

Reference Tasks Study Guide & Solutions

This guide provides the solutions and explanations for **Task 8: Parallel Streaming Data Processor** and **Task 14: The Print Server Queue**. These questions represent typical mock exams that test your understanding of the **Master-Worker (Farm) pattern in Python** and **Semaphore-based synchronization in C/Pthreads**.

Task 8: Parallel Streaming Data Processor (Python)

Goal: Implement a Master-Worker (Farm) skeleton to process a continuous stream of data concurrently. We must use a generator that produces integers every millisecond and distribute their square calculation across at least 4 worker processes/threads.

Concepts Tested:

- **Farm Pattern (Master-Worker Skeleton):** A master node distributes tasks to worker nodes, which process them independently and return the results.
- **Inter-Process Communication (IPC):** Using thread-safe or process-safe queues ensures workers don't interfere with each other and avoid race conditions.

Solution Code (Using `multiprocessing` and Queues)

This implementation explicitly defines a Master that consumes the stream and pushes to a `task_queue`, and 4 Workers that compute the squares and push to a `result_queue`.

```
import time
import random
import multiprocessing as mp

# Provided Generator
def data_stream(n=50):
    for i in range(n):
        value = random.randint(1, 100)
        time.sleep(0.001) # Simulate work arrival time
        yield (value, time.time())

# Worker Process
def worker(task_queue, result_queue):
    while True:
        task = task_queue.get()
        if task is None: # Poison pill to stop the worker
            break
        value, timestamp = task
        result = value ** 2
        # Return the original value, squared result, and time it took
        result_queue.put((value, result, time.time() - timestamp))

# Master Process
def master():
```

```

task_queue = mp.Queue()
result_queue = mp.Queue()

num_workers = 4
workers = []

# 1. Start workers
for _ in range(num_workers):
    p = mp.Process(target=worker, args=(task_queue, result_queue))
    p.start()
    workers.append(p)

# 2. Master consumes data stream and dispatches tasks
total_tasks = 50
for data in data_stream(total_tasks):
    task_queue.put(data)

# 3. Master collects results (can be done asynchronously in a real app)
for _ in range(total_tasks):
    val, sq_val, latency = result_queue.get()
    print(f"[Master] Result received: {val}^2 = {sq_val} (Latency: {latency:.4f}s)")

# 4. Send poison pills to gracefully terminate workers
for _ in range(num_workers):
    task_queue.put(None)

for p in workers:
    p.join()

print("All tasks processed successfully.")

if __name__ == "__main__":
    master()

```

Key modifications to memorize for exams:

- **Poison Pill Technique:** Using `task_queue.put(None)` is the standard way to tell worker loops to terminate gracefully.
- **Generators:** You iterate over `data_stream()` directly to put items in the Queue without loading the whole stream into memory.

Task 14: The Print Server Queue (C Pthreads)

Background: A company has a shared print server that can process up to 3 print jobs simultaneously. The provided program incorrectly tracks `active_jobs` with a simple `if` check, which leads to race conditions (no mutex) and busy-waiting/skipping jobs instead of waiting efficiently.

Concepts Tested:

- **Counting Semaphores:** The ideal synchronization primitive when you have multiple identical resources (in this case, 3 print slots). It allows up to N threads to proceed, putting the rest to sleep efficiently.
- **Race Conditions:** `active_jobs++` is not atomic.
- **Thread Joining:** Standard pthread lifecycle.

Solution Code (Using `<semaphore.h>`)

We replace the `active_jobs` integer checks with a `sem_t` initialized to 3.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>

// Initialize a counting semaphore to limit concurrency
sem_t print_slots;

void* print_job(void* arg) {
    long thread_id = (long)arg;

    for (int i = 0; i < 5; i++) {
        // Wait efficiently until a slot opens up (semaphore value > 0)
        sem_wait(&print_slots);

        // --- Critical Section: Taking a slot ---
        // We can get the current value of the semaphore purely for logging
        (optional)
        int active_slots;
        sem_getvalue(&print_slots, &active_slots);

        printf("Thread %ld printing... Remaining slots: %d\n", thread_id,
active_slots);
        sleep(1); // simulate printing time

        // --- Free the slot ---
        sem_post(&print_slots);
    }
    return NULL;
}

int main() {
    pthread_t t[6];

    // Initialize semaphore: shared between threads (0), initial value 3
    if (sem_init(&print_slots, 0, 3) != 0) {
        perror("Semaphore init failed");
        return 1;
    }

    for (long i = 0; i < 6; i++) {
        pthread_create(&t[i], NULL, print_job, (void*)i);
    }
}
```

```

    }

    for (int i = 0; i < 6; i++) {
        pthread_join(t[i], NULL);
    }

    // Destroy semaphore
    sem_destroy(&print_slots);

    return 0;
}

```

Why this is the correct solution:

1. **"No more than three run at the same time"**: `sem_init(&print_slots, 0, 3)` exactly fulfills this.
2. **"Wait efficiently when all slots are full"**: `sem_wait()` blocks the thread efficiently without spinning the CPU when the semaphore value drops to 0.
3. **Removes race conditions**: The semaphore operations `sem_wait` and `sem_post` are inherently atomic, making the broken `active_jobs++` logic obsolete.

Combined / Mixed-Concept Mock Exam Questions

Exams often test your ability to synthesize concepts from multiple labs. Below are complex, combined-concept questions with step-by-step solutions mapping Skeletons, Threading, SIMD, and OpenMP together.

Combined Task 1: Map-Reduce Using OpenMP (Lab 5 + Lab 6)

Goal: You are given an array of 500,000 integers. You need to square each integer (Map Skeleton) and then sum all the squared values together (Reduce Skeleton). Implement this efficiently in C using OpenMP.

Concepts Tested:

- Skeletons (Map and Reduce) mapped to OpenMP directives.
- OpenMP `parallel for` and `reduction` clauses.

Step-by-Step Solution:

1. **Identify the Map operation:** Squaring each element is an embarrassingly parallel operation. We can use an OpenMP `for` loop to distribute the iterations.
2. **Identify the Reduce operation:** Summing the results requires an accumulator, which is susceptible to race conditions.
3. **Combine them:** OpenMP allows combining these in a single loop using the `reduction(+:sum)` clause.

```

#include <stdio.h>
#include <omp.h>

#define SIZE 500000

int main() {

```

```

    long long data[SIZE];
    long long total_sum_of_squares = 0;

    // Initialize array sequentially
    for (int i = 0; i < SIZE; i++) {
        data[i] = i % 100;
    }

    // Step 1 & 2: Map and Reduce in a single parallel loop
    #pragma omp parallel for reduction(+:total_sum_of_squares) schedule(static)
    for (int i = 0; i < SIZE; i++) {
        // Map step: Square the number locally
        long long squared = data[i] * data[i];

        // Reduce step: Add to total (OpenMP handles the local variable merging
        // automatically)
        total_sum_of_squares += squared;
    }

    printf("Total Sum of Squares: %lld\n", total_sum_of_squares);
    return 0;
}

```

Why this works: We avoid creating an intermediate array to store the squared values. By keeping the mapping and reduction inside the same loop, we save memory bandwidth and execute at peak efficiency.

Combined Task 2: Pthreads Pipeline Synchronization (Lab 2 + Lab 5)

Goal: Implement a 3-stage Pipeline skeleton (Read -> Process -> Write) using C Pthreads. Stage A reads lines from a file and passes them to Stage B. Stage B capitalizes the text and passes to Stage C. Stage C writes to a new file.

Concepts Tested:

- Pipeline Skeleton (Lab 5)
- Producer-Consumer Synchronization (Lab 2) using Mutexes and Semaphores.

Step-by-Step Solution:

1. **Shared Buffers:** We need two buffers (Buffer AB between Stage A and B, and Buffer BC between Stage B and C).
2. **Synchronization primitives:** Each buffer needs a `pthread_mutex_t` to protect its data and two `sem_t` (semaphores) to track empty slots and full slots.
3. **Stage Logic Flow:**
 - **Stage A (Producer to AB):** `sem_wait(&empty_AB)`, lock mutex, write data, unlock mutex, `sem_post(&full_AB)`.
 - **Stage B (Consumer from AB, Producer to BC):** `sem_wait(&full_AB)`, lock mutex, read data, unlock mutex, `sem_post(&empty_AB)`. Then process data. Then `sem_wait(&empty_BC)`, lock mutex, write data, unlock mutex, `sem_post(&full_BC)`.

```
// Pseudo-code snippet for Stage B (The Middleman in the Pipeline)
void* stage_B(void* arg) {
    while (1) {
        // --- CONSUME FROM BUFFER AB ---
        sem_wait(&full_slots_AB);           // Wait for data to exist
        pthread_mutex_lock(&mtx_AB);        // Lock buffer
        char* data = read_from_buffer_AB();
        pthread_mutex_unlock(&mtx_AB);
        sem_post(&empty_slots_AB);          // Signal that we took an item

        if (data == NULL) break;            // End of stream (Poison pill)

        // --- PROCESS (Map Skeleton applied continuously) ---
        capitalize_text(data);

        // --- PRODUCE TO BUFFER BC ---
        sem_wait(&empty_slots_BC);           // Wait for space in next buffer
        pthread_mutex_lock(&mtx_BC);
        write_to_buffer_BC(data);
        pthread_mutex_unlock(&mtx_BC);
        sem_post(&full_slots_BC);           // Signal that we added an item
    }

    // Pass the poison pill forward to Stage C
    sem_wait(&empty_slots_BC);
    pthread_mutex_lock(&mtx_BC);
    write_to_buffer_BC(NULL);
    pthread_mutex_unlock(&mtx_BC);
    sem_post(&full_slots_BC);

    return NULL;
}
```

Key Takeaway: Pipelines require careful buffer management to avoid deadlocks. The **empty** and **full** semaphore pairing is the standard design pattern for any producer-consumer problem.

Combined Task 3: SIMD + OpenMP Hybrid (Lab 4 + Lab 6)

Goal: You need maximum performance for computing a vector addition $C = A + B + A$ on a massive array of length 10^8 . You have an 8-core CPU that supports AVX2.

Concepts Tested:

- Combining thread-level parallelism (OpenMP) with instruction-level parallelism (SIMD).
- Thread scheduling for heavy memory operations.

Step-by-Step Solution:

1. **Thread-Level Distribution:** Use OpenMP to divide the 10^8 array into 8 sequential chunks of size 1.25×10^7 .

2. **Instruction-Level Execution:** Inside each thread's chunk, use an AVX2 FMA (Fused Multiply-Add) loop to process 8 floats per clock cycle instead of 1 float.

```
#include <omp.h>
#include <immintrin.h>

void hybrid_vector_op(float* a, float* b, float* c, int size) {
    // 1. Thread Parallelism
    // Static schedule is best here because the work is perfectly uniform across
    // array indexes
    #pragma omp parallel for schedule(static)
    for (int block = 0; block <= size - 8; block += 8) {

        // 2. SIMD Parallelism inside the thread
        __m256 vec_a = _mm256_loadu_ps(&a[block]);
        __m256 vec_b = _mm256_loadu_ps(&b[block]);

        // FMA computes A*B + C. To get A+B+A, we can compute (A*1) + B ... wait,
        // it's simpler
        // Let's just do C = A + B + A
        __m256 vec_c = _mm256_add_ps(vec_a, vec_b);
        vec_c = _mm256_add_ps(vec_c, vec_a);

        _mm256_storeu_ps(&c[block], vec_c);
    }

    // Tail loop (runs for the last few elements, < 8 loops)
    // Could manually assign this to the master thread but it's trivially small
    int tail_start = size - (size % 8);
    for (int i = tail_start; i < size; i++) {
        c[i] = a[i] + b[i] + a[i];
    }
}
```

Why this is the ultimate optimization: OpenMP keeps all 8 cores active. Within each core, SIMD keeps the ALUs fully saturated by processing 256 bits of data simultaneously. You get a theoretical $8 \times 8 \times 8 = 64$ speedup!